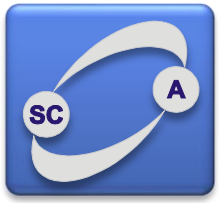


UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA
KATEDRA ZA AUTOMATIKU I UPRAVLJANJE SISTEMIMA

Simulacija:
Model opisan običnim diferencijalnim jednačinama

Modeliranje i simulacija sistema

Upravljanje, modelovanje i simulacija sistema

Obične diferencijalne jednačine (ODJ)

- engl. *Ordinary Differential Equation* (ODE)
- U matematici ODJ je jednačina koju čini funkcija jedne nezavisne promenljive i njenih izvoda.
- Veoma su česte u modelovanju dinamičkih sistema
  - U simulacijama nezavisno promenljiva je vreme *t*

- Generalna definicija ODJ:

$$y^{(n)} = f(t, y, y', y'', y''', \ldots . y^{(n-1)})$$

Eksplicitna forma

ili

$$f(t, y, y', y'', y''', \ldots, y^{(n-1)}, y^{(n)}) = 0$$

Implicitna forma

gde je:

$$y' = \frac{dy}{dt}, y'' = \frac{d^2y}{dt^2}, \ldots, y^{(n)} = \frac{d^ny}{dt^n}$$

Parcijalne diferencijalne jednačine su složenije od ODJ jer sadrže izvode više nezavisnih promenljivih. ⇦ Ne razmatramo.

Linearna ODJ

- Linearna ODJ sadrži linearnu kombinaciju izvoda zavisno promenljive y

$$y^{(n)} = \sum_{i=0}^{n-1} a_i(t) \cdot y^{(i)} + r(t)$$

ili

$$y^{(n)} = a_{n-1}y^{(n-1)} + \ldots + a_1 y' + a_0 y + r(t)$$

- Linearne ODJ se rešavaju na poseban način koji je dosta jednostavniji od opšteg postupka.

Svođenje ODJ na sistem DJ 1. reda

- Svaka ODJ se može napisati kao ekvivalentan sistem običnih diferencijalnih jednačina 1. reda.
- Jedan od načina za takvu transformaciju je uvođenje smena

$$y_k = y^{(k-1)}, \quad k = 1,2,\dots,n$$

- Dobija se:

$$\begin{aligned} y_1' &= y_2 \\ y_2' &= y_3 \\ &\dots \\ y_{n-1}' &= y_n \\ y_n' &= f(t, y_1, \dots, y_n) \end{aligned}$$

ili zapisano u vektorskom obliku:

$$\boldsymbol{y}' = \boldsymbol{f}(t, \boldsymbol{y})$$

$$\boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_n \end{bmatrix}, \boldsymbol{y}' = \begin{bmatrix} y_1' \\ y_2' \\ \dots \\ y_n' \end{bmatrix}, \boldsymbol{f} = \begin{bmatrix} f_1 \\ f_2 \\ \dots \\ f_n \end{bmatrix} = \begin{bmatrix} y_2 \\ y_3 \\ \dots \\ f(t, y_1, \dots, y_n) \end{bmatrix}$$

Primer transformacije

- Posmatra se ODJ:

$$\frac{d^2y}{dt^2} - \mu(1 - y^2)\frac{dy}{dt} + y = 0$$

Smene:

$$y_1 = y$$
$$y_2 = \frac{dy}{dt}$$

$\xrightarrow{\frac{d}{dt}}$

$$\frac{dy_1}{dt} = \frac{dy}{dt}$$
$$\frac{dy_2}{dt} = \frac{d^2y}{dt^2}$$

$\longrightarrow$

$$\frac{dy_1}{dt} = y_2$$
$$\frac{dy_2}{dt} = \mu(1 - {y_1}^2)y_2 - y_1$$

Uopšten sistem DJ 1. reda

- Sistem n običnih diferencijalnih jednačina prvog reda, gde se pojavljuje n zavisno promenljivih $y_1, \dots, y_n$

$$\frac{dy_1}{dt} = f_1(y_1, y_2, \dots, y_n, t), \quad y_1(t_0) = y_{10}$$
$$\frac{dy_2}{dt} = f_2(y_1, y_2, \dots, y_n, t), \quad y_2(t_0) = y_{20}$$
$$\dots$$
$$\frac{dy_n}{dt} = f_n(y_1, y_2, \dots, y_n, t), \quad y_n(t_0) = y_{n0}$$

Numeričko rešavanje ODJ

- Svodi se na rešavanje sistema diferencijalnih jednačina 1. reda
- Šire posmatrano, sistem običnih diferencijalnih jednačina se takođe svodi na sistem diferencijalnih jednačina 1. reda

- Primer:

$$m_1 x_1'' + c_1 x_1' + k_1 x_1 + k(x_1 - x_2) = f(t)$$
$$m_2 x_2'' + c_2 x_2' + k_2 x_2 + k(x_2 - x_1) = 0$$

$$y_1 = x_1 \qquad y_1' = x_1' = y_2$$
$$y_2 = x_1' \qquad y_2' = x_1'' = \frac{1}{m_1}(f(t) - c_1 y_2 - k_1 y_1 - k(y_1 - y_3))$$
$$y_3 = x_2 \qquad y_3' = x_2' = y_4$$
$$y_4 = x_2' \qquad y_4' = x_2'' = \frac{1}{m_2}(-c_2 y_4 - k_2 y_3 - k(y_3 - y_1))$$

- (Bez gubitka na opštosti) nadalje se posmatra rešavanje jedne DJ 1. reda.

Jednostavniji problem …

- Problem:

$$\frac{dy}{dt} = f(t), \quad y(t_0) = y_0$$

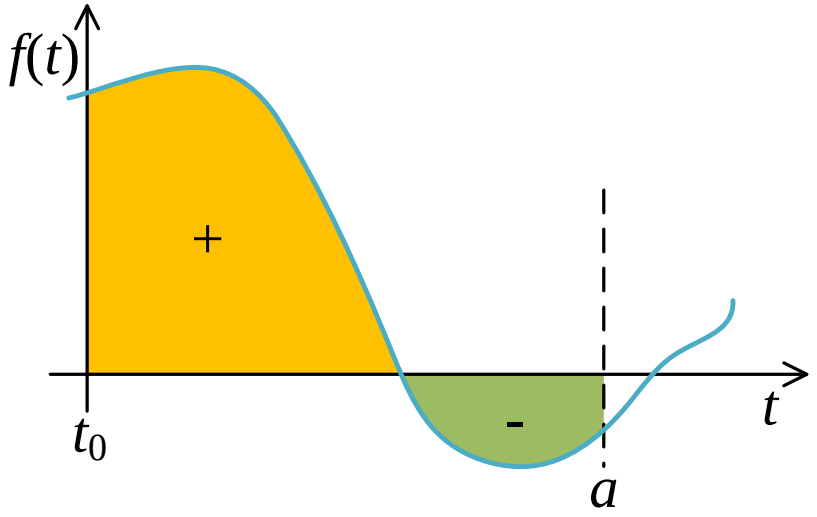
ima analitičko rešenje:

$$y(t) = y_0 + \int_{t_0}^{t} f(t)dt$$

- Konkretna vrednost

$$y(a) = \int_{0}^{a} f(t)dt\,, \quad t_0 = 0$$

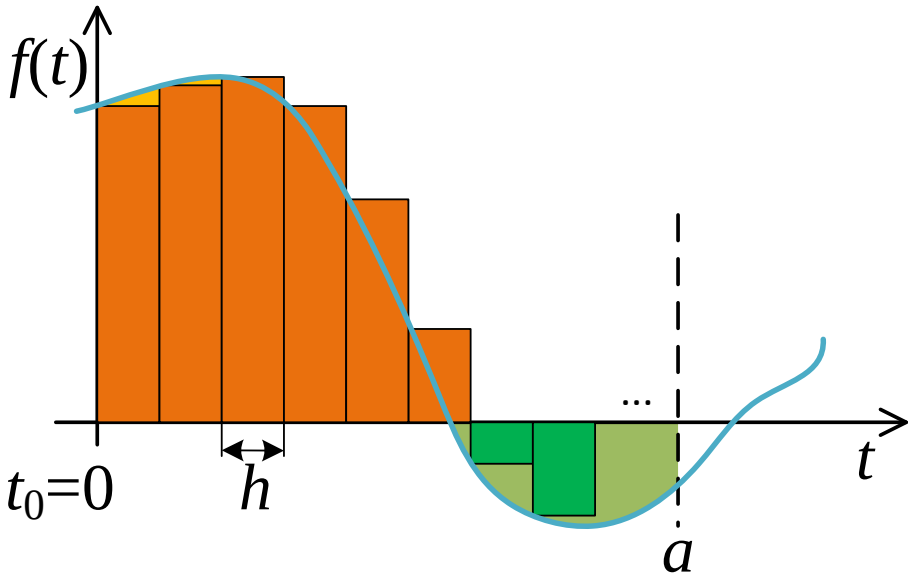
predstavlja površinu sa slike



Ideja numeričkog računanja

- Odredimo vrednosti funkcije $f(t)$ u diskretnim trenucima $i \cdot h, i = 0,1,2, \ldots$
- Vrednost integrala se može aproksimirati sumom pravougaonika sa slike

$$y(a) \approx \sum_{i=0}^{N-1} f(ih) \cdot h$$



- Ovakva računica je veoma gruba i **praktično se ne koristi**.
- Ako se h smanji greška se smanjuje, ali se produžava vreme računanja.

Numeričko rešavanje DJ 1. reda

Problem:

$$\frac{dy}{dt} = f(y,t), \quad y(t_0) = y_0$$

Važno:
Zna se $\frac{dy}{dt}$, a traži se $y(t)$

Rešenje:

- polazi se od poznatih (t_0, y_0) i sukcesivno računaju parovi (t_i, y_i) za $i = 1,2, \ldots$
- Iterativan račun se sprovodi "korak-po-korak", gde se (t_{i+1}, y_{i+1}) računa na osnovu prethodno sračunatog (t_i, y_i).

- Ovde će se posmatrati:
  1. Ojlerov postupak, i
  2. Metode Runge-Kuta

Leonhard Euler

Born	15 April 1707 Basel, Switzerland
Died	18 September 1783 (aged 76) [OS: 7 September 1783] Saint Petersburg, Russian Empire
Residence	Kingdom of Prussia, Russian Empire Switzerland
Nationality	Swiss
Fields	Mathematics and physics
Institutions	Imperial Russian Academy of Sciences Berlin Academy
Alma mater	University of Basel
Doctoral advisor	Johann Bernoulli
Doctoral students	Nicolas Fuss Johann Hennert Joseph Louis Lagrange Stepan Rumovsky

Ojlerov postupak

- Vrednost y_{i+1} se može odrediti na osnovu razvoja u Tejlorov red u okolini tačke (t_i, y_i)

$$y_{i+1} = y_i + \Delta y = y_i + \frac{dy_i}{dt}h + \frac{d^2y_i}{dt^2}\frac{h^2}{2!} + \ldots$$

$$h = t_{i+1} - t_i$$

- Za h malo y_{i+1} se aproksimira sa

$$y_{i+1} \approx y_i + \frac{dy_i}{dt}h$$

Primetiti da $\frac{dy}{dt} = f(t, y)$ znamo.

- Nakon diferenciranja se dobija

$$\frac{dy_{i+1}}{dt} = \frac{dy_i}{dt} + \frac{d^2y_i}{dt^2}h \quad \text{ili} \quad \frac{d^2y_i}{dt^2} = \left(\frac{dy_{i+1}}{dt} - \frac{dy_i}{dt}\right)\frac{1}{h}$$

- Smenom u gornji izraz se dobija bolja aproksimacija

$$y_{i+1} = y_i + \frac{dy_i}{dt}h + \frac{d^2y_i}{dt^2}\frac{h^2}{2!} = y_i + \frac{dy_i}{dt}h + \left(\frac{dy_{i+1}}{dt} - \frac{dy_i}{dt}\right)\frac{h}{2}$$

Ojlerov postupak (2)

U svakoj iteraciji :

$$y_{i+1} = y_i + \frac{dy_i}{dt}h + \left(\frac{dy_{i+1}}{dt} - \frac{dy_i}{dt}\right)\frac{h}{2}$$

Računanje izvoda traži vrednost y_{i+1} koju još ne znamo!

1. Uvodi se predviđena (*predicted*) vrednost

$$y_{i+1}^p = y_i + \frac{dy_i}{dt}h$$

metod 1. reda (ako se ne radi drugi korak)

2. I ona se koriguje

$$\varepsilon_i = \left(\frac{dy_{i+1}^p}{dt} - \frac{dy_i}{dt}\right)\frac{h}{2}$$

Korekcija, a ujedno i procena greške

$$y_{i+1}^c = y_{i+1}^p + \varepsilon_i$$

metod 2. reda

Ojlerov postupak - realizacija

U tekućoj iteraciji …

- indeks $i+1$ je zamenjen sa 1
- prethodna iteracija ima indeks 0

$$\frac{dy_0}{dt} = f(t_0, y_0)$$

$$y_1^p = y_0 + \frac{dy_0}{dt} h$$

$$\frac{dy_1^p}{dt} = f\left(t_1, y_1^p\right)$$

$$\varepsilon = \left(\frac{dy_1^p}{dt} - \frac{dy_0}{dt}\right)\frac{h}{2}$$

$$y_1^c = y_1^p + \varepsilon$$

```
function OjlerKorak(f,t0,y0,h)
    dy0 = f(t0,y0);              # izvod u tački (t0,y0)
    y1p = y0 + dy0*h;
    t1 = t0 + h;
    dy1 = f(t1,y1p);             # izvod u tački (t1,y1p)
    e = (dy1-dy0)*h/2.0;         # procena greške
    y1c = y1p + e;               # korekcija
    return t1,y1c,e
end
```

Rešenje nad intervalom $[0, t_k]$ sa korakom h …

```
function Ojler2(f,tk,y0,h)
    t0 = 0;
    Y = [y0];
    koraka = tk / h;
    for i = 1:koraka
        t1, y1, e = OjlerKorak(f,t0,y0,h);
        y0 = y1;
        t0 = t1;
        push!(Y, y1);     # pamti se y1
    end
    return Y
end
```

Primer: DJ 1. reda

Problem:

$$\frac{dy}{dt} = \lambda \cdot e^{-\alpha \cdot t} \cdot y$$
$$y(0) = y_0$$

```
function difJedn(t,y)
    # opis DJ
    α = 1; λ = 1;
    dy = λ*exp(-α*t)*y;
    return dy
end
```

```
function analitickoResenje(t,y0)
    # analitičko rešenje
    α = 1; λ = 1;
    y = y0*exp(λ/α*(1-exp(-α*t)));
    return y
end
```

Analitičko rešenje:

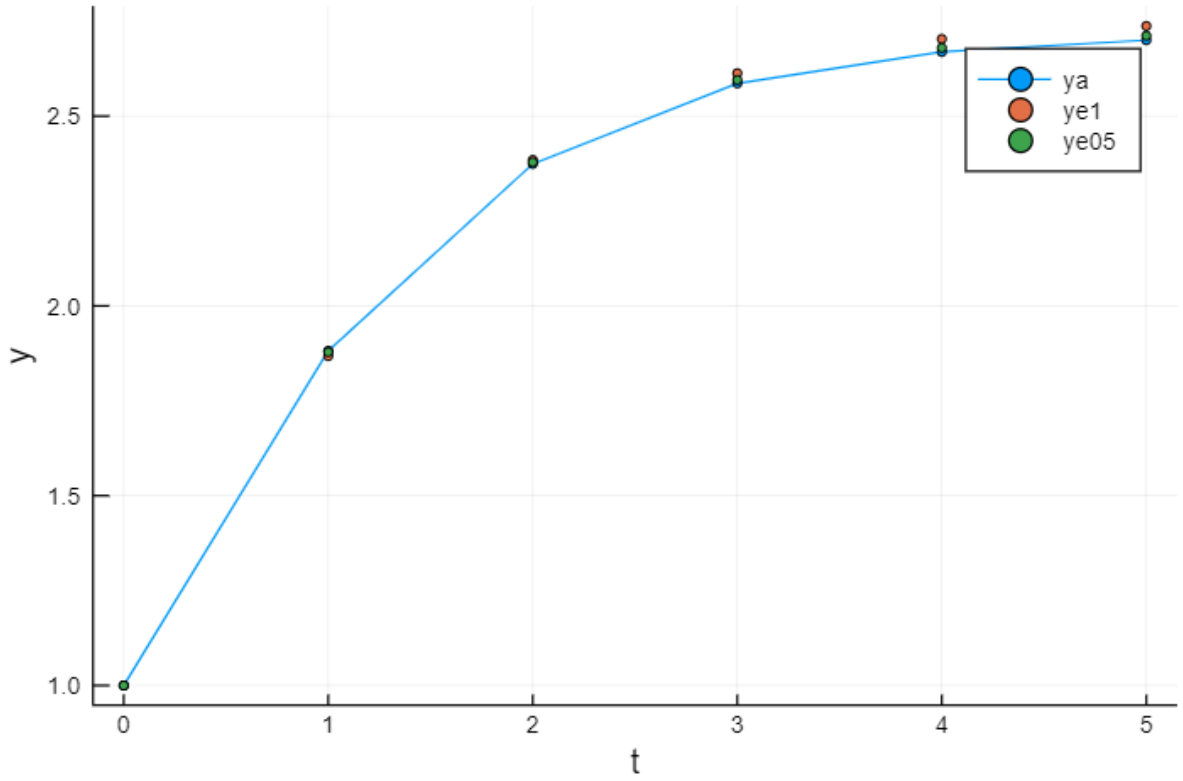
$$y(t) = y_0 \cdot e^{\frac{\lambda}{\alpha} \cdot (1 - e^{-\alpha \cdot t})}$$

```
tkraj = 5;
t = (0:tkraj);
y0 = 1.0;
ya = analitickoResenje.(t,y0);

y1  = Ojler2( difJedn, tkraj, y0, 1 );      # h=1
y05 = Ojler2( difJedn, tkraj, y0, 0.5 );    # h=0.5

y05 = y05[1:2:end];       # izbaci rešenja u 0.5s, 1.5s, …
[t ya y1 y1-ya y05 y05-ya]
```

Primer: rezultati

t	ya	ye1	ye1-ya	ye05	ye05-ya
0.0	1.0000	1.0000	0	1.0000	0
1.0	1.8816	1.8679	-0.0137	1.8786	-0.0030
2.0	2.3742	2.3843	0.0101	2.3786	0.0044
3.0	2.5863	2.6131	0.0268	2.5952	0.0090
4.0	2.6689	2.7033	0.0343	2.6799	0.0110
5.0	2.7000	2.7373	0.0373	2.7118	0.0117

Legenda:
ya – analitičko rešenje
ye1 – Euler 2. reda sa fiksnim korakom h=1
ye05 – Euler 2. reda sa fiksnim korakom h=0.5



Promenljivi (adaptivni) korak integracije

- Računanje korekcije ε zavisi od koraka h

$$\varepsilon_i = \left(\frac{dy_{i+1}^{p}}{dt} - \frac{dy_i}{dt} \right) \frac{h}{2}$$

- Ideja: na osnovu veličine ε prilagoditi korak h
  - za veliko ε smanjiti korak, a
  - za malo ε povećati korak

- Parametri proračuna:
  - Dozvoljena relativna greška *relGr*
  - Dozvoljena apsolutna greška *apsGr*
  - Minimalan korak h_{min}

Implementacija:
Promenljivi korak integracije (2)

„Zanimljivo“ računanje dozvoljene greške:

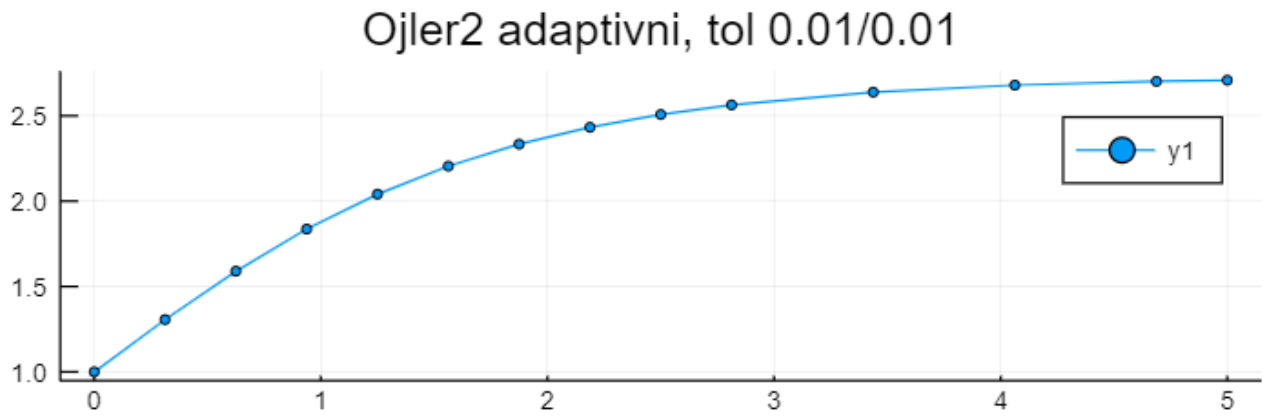
$$|y| \cdot gr_{rel} + gr_{aps}$$

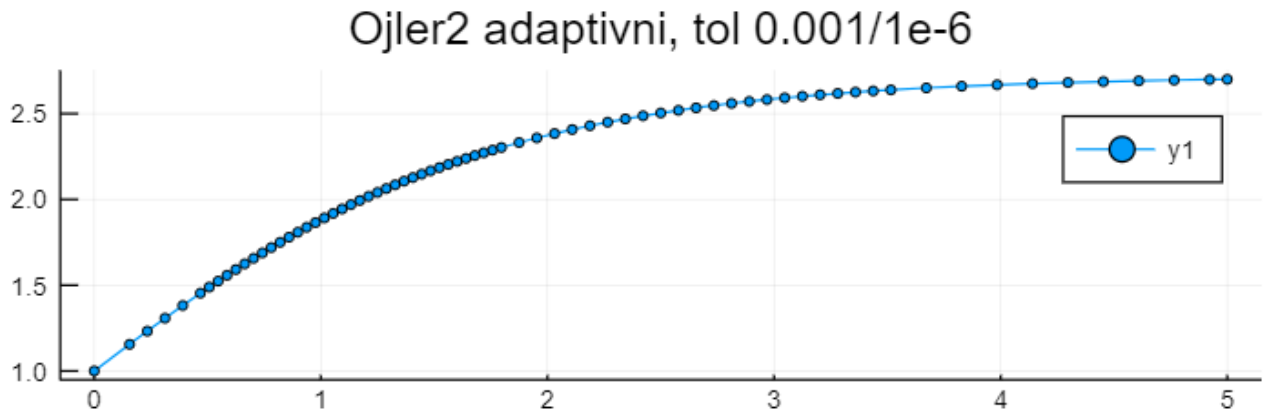
```
function Ojler2ad(f,tk,y0,hmin,apsGr,relGr)
    t0 = 0.0;
    Y = [y0]; T = [t0];
    h = (tk - t0)/8.0;           # početni korak

    while t0 < tk
        if t0+h > tk
            h = tk - t0;         # ograniči poslednji korak
        end
        t1, y1, e = OjlerKorak(f,t0,y0,h);
        korakDobar = true;       # fleg za kraj koraka integracije

        if h > hmin              # Ako je greška velika smanji korak i resetuj fleg
            if abs(e) > abs(y1)*relGr+apsGr
                h = h / 2.0;
                korakDobar = false;
            end
            if h < hmin
                h = hmin;        # ograniči min korak
            end
        end

        if korakDobar            # Nema greške: nastavi ka novoj tacki
            y0 = y1; t0 = t1;
            push!(Y, y1);
            push!(T, t1);
            # da li se korak može povećati?
            if abs(e) < (abs(y1)*relGr+apsGr)/4.0
                h = h * 2.0;
            end
        end
    end
    return T, Y
end
```

Primer: rezultati sa promenljivim korakom

```
tkraj = 5;
t = (0:tkraj);
y0 = 1.0;
ya = analitickoResenje.(t,y0);

t3, y3 = Ojler2ad(difJedn,tkraj,y0,0.01,0.01,0.01);
t4, y4 = Ojler2ad(difJedn,tkraj,y0,0.01,0.001,0.000001);

#…
```

function Ojler2ad(f,tk,y0,hmin,apsGr,relGr)





t	ya	y3i	y3i-ya	y4i	y4i-ya
0.0	1.0000	1.0000	0	1.0000	0
1.0	1.8816	1.8773	-0.0043	1.8811	-0.0005
2.0	2.3742	2.3732	-0.0010	2.3736	-0.0006
3.0	2.5863	2.5862	-0.0001	2.5858	-0.0004
4.0	2.6689	2.6755	0.0065	2.6686	-0.0003
5.0	2.7000	2.7085	0.0085	2.6998	-0.0002

Legenda:
ya – analitičko rešenje
y3i – Euler 2. reda sa promenljivim korakom: h=0.01 relGr=0.01 apsGr=0.01
y4i – Euler 2. reda sa promenljivim korakom: h=0.01 relGr=0.001 apsGr=0.000001

Metode Runge-Kuta (RK)

- Runge i Kuta su uopštili Ojlerov postupak
- Pored toga, uveli su više članova koji bolje aproksimiraju traženu vrednost za y
  - Ako se koristi 2 člana onda se govori o metodama 2. reda (postoji čitava familija tih metoda)
  - Ako se koristi 3 člana onda se govori o metodama 3. reda (opet, više metoda)
  - …

Carl Runge

Born	30 August 1856 Bremen, German Confederation
Died	3 January 1927 (aged 70) Göttingen, Weimar Republic
Residence	Germany
Citizenship	German
Fields	Mathematics Physics
Institutions	University of Hannover (1886–1904) Georg-August University of Göttingen (1904–1925)
Alma mater	Berlin University
Doctoral advisor	Karl Weierstrass Ernst Kummer
Doctoral students	Max Born
Known for	Runge–Kutta method Runge's phenomenon Laplace–Runge–Lenz vector

Martin Wilhelm Kutta

Born	November 3, 1867 Pitschen, Upper Silesia
Died	December 25, 1944 (aged 77) Fürstenfeldbruck
Residence	Germany
Nationality	German
Fields	Mathematician
Institutions	University of Stuttgart RWTH Aachen
Alma mater	University of Breslau University of Munich
Doctoral advisor	C. L. Ferdinand Lindemann Gustav A. Bauer
Other academic advisors	Walther Franz Anton von Dyck
Known for	Runge-Kutta method Zhukovsky-Kutta aerofoil Kutta–Joukowski theorem Kutta condition

RK 2. reda

- Ojlerov metod – ponovo …

$$y_{i+1}^{p} = y_i + f(y_i, t_i)h = y_i + k_1$$

$$\varepsilon_i = (f(y_i + k_1, t_i + h) - f(y_i, t_i))\frac{h}{2} = \frac{k_2 - k_1}{2}$$

$$y_{i+1}^{c} = y_i + k_1 + \frac{k_2 - k_1}{2} = y_i + \frac{1}{2}k_1 + \frac{1}{2}k_2$$

- Može se zapisati kao

$$y_{i+1} = y_i + \frac{1}{2}k_1 + \frac{1}{2}k_2$$

$$k_1 = f(y_i, t_i)h$$

$$k_2 = f(y_i + k_1, t_i + h)h$$

RK uvodi tri konstante c_1, c_2, a_2

$$y_{i+1} = y_i + c_1k_1 + c_2k_2$$

$$k_1 = f(y_i, t_i)h$$

$$k_2 = f(y_i + a_2k_1, t_i + a_2h)h$$

Pretpostavka: $y_{i+1} = y_i + c_1 k_1 + c_2 k_2$ $\quad k_1 = f(y_i, t_i)h$ $\quad k_2 = f(y_i + a_2 k_1, t_i + a_2 h)h$

Aproksimacija k_2

$$f(y_i + \Delta y, t_i + \Delta t) = f(y_i, t_i) + \frac{\partial f(y_i, t_i)}{\partial y}\Delta y + \frac{\partial f(y_i, t_i)}{\partial t}\Delta t + \dots$$

$$\Delta y = a_2 k_1$$
$$\Delta t = a_2 h$$

$$k_2 = \left(f(y_i, t_i) + \frac{\partial f(y_i, t_i)}{\partial y} a_2 f(y_i, t_i)h + \frac{\partial f(y_i, t_i)}{\partial t} a_2 h + O(h^2) \right) h$$

Smena u y_{i+1}

$$y_{i+1} = y_i + c_1 f(y_i, t_i)h + c_2 \left(f(y_i, t_i) + \frac{\partial f(y_i, t_i)}{\partial y} a_2 f(y_i, t_i)h + \frac{\partial f(y_i, t_i)}{\partial t} a_2 h \right) h + O(h^3)$$

$$y_{i+1} = y_i + (c_1 + c_2) f(y_i, t_i)h + c_2 a_2 \underbrace{\left(\frac{\partial f(y_i, t_i)}{\partial y} f(y_i, t_i) + \frac{\partial f(y_i, t_i)}{\partial t} \right)} h^2 + O(h^3)$$

„pomoć“ → $\overbrace{\frac{df(y_i, t_i)}{dt} = \frac{\partial f(y_i, t_i)}{\partial y}\frac{\partial y}{\partial t} + \frac{\partial f(y_i, t_i)}{\partial t} = \frac{\partial f(y_i, t_i)}{\partial y} f(y_i, t_i) + \frac{\partial f(y_i, t_i)}{\partial t}}$

$$y_{i+1} = y_i + (c_1 + c_2) f(y_i, t_i)h + c_2 a_2 \frac{df(y_i, t_i)}{dt} h^2 + O(h^3)$$

A treba da bude $\quad y_{i+1} = y_i + \frac{dy_i}{dt}h + \frac{d^2 y_i}{dt^2}\frac{h^2}{2!} + O(h^3)$ ← Odavde se krenulo

Znači: $\quad c_1 + c_2 = 1, \qquad c_2 a_2 = \frac{1}{2}$

RK 2. reda

- Može se pokazati da između c_1, c_2, a_2 vrednosti postoje zavisnosti (izvođenje na prethodnom slajdu)

$$c_1 + c_2 = 1, \qquad c_2 a_2 = \frac{1}{2}$$

- Ovaj sistem 3 jednačine sa 2 nepoznate daje slobodu izbora rešenja.

- Jedno moguće rešenje vodi ka Ojlerovom metodu 2. reda

$$c_1 = \frac{1}{2}, \quad c_2 = \frac{1}{2}, \quad a_2 = 1$$

- Postoje i druga rešenja, npr.

$$c_1 = 0, \quad c_2 = 1, \quad a_2 = \frac{1}{2} \qquad \text{ili} \qquad c_1 = \frac{1}{4}, \quad c_2 = \frac{3}{4}, \quad a_2 = \frac{2}{3} \qquad \text{ili ...}$$

- U tim rešenjima se vrednosti f računaju unutar intervala h.

Familija metoda RK 2. reda

RK 3. reda

- Uvode se konstante: $c_1, c_2, c_3, a_2, a_3, b_3$

$$y_{i+1} = y_i + c_1 k_1 + c_2 k_2 + c_3 k_3$$
$$k_1 = f(y_i, t_i)h$$
$$k_2 = f(y_i + a_2 k_1, t_i + a_2 h)h$$

} Isto kao kod RK 2. reda

$$k_3 = f(y_i + b_3 k_1 + (a_3 - b_3)k_2, t_i + a_3 h)h$$

- Veze konstanti

$$c_1 + c_2 + c_3 = 1$$
$$c_2 a_2 + c_3 a_3 = \frac{1}{2}$$
$$c_2 a_2^2 + c_3 a_3^2 = \frac{1}{3}$$
$$c_3 (a_3 - b_3) a_2 = \frac{1}{6}$$

Moguće rešenje:

$$c_1 = \frac{2}{8}$$
$$c_2 = c_3 = \frac{3}{8}$$
$$a_2 = a_3 = \frac{2}{3}$$
$$b_3 = 0$$

Familija metoda RK 3. reda

ili ... ili ...

RK 3. reda - realizacija

$$k_1 = f(y_i, t_i)h$$

$$k_2 = f\left(y_i + \frac{2}{3}k_1, t_i + \frac{2}{3}h\right)h$$

$$k_3 = f(y_i + \frac{2}{3}k_2, t_i + \frac{2}{3}h)h$$

$$y_{i+1} = y_i + \frac{1}{4}k_1 + \frac{3}{8}k_2 + \frac{3}{8}k_3$$

```
function rkutta3( F, tp, tf, y0, h )
    # Metod integracije RUNGE-KUTTA 3. reda sa nepromenljivim korakom

    T = tp : h : tf;
    n = length(T);
    Y = zeros( n, length(y0) );
    Y[1,:] = y0[:]';
    t = tp;
    for i = 1 : n-1
        y = Y[i,:]';
        k1 = h * F( t, y )
        k2 = h * F( t+2/3*h, y+2/3*k1 );
        k3 = h * F( t+2/3*h, y+2/3*k2 );
        Y[i+1,:] = (y + k1/4 + 3/8*k2 + 3/8*k3)';
        t += h;
    end
    return T, Y
end
```

Ova realizacija se može upotrebiti za problem sa više zavisno promenljivih $y_1, \ldots, y_n$.
Tada je F vektor (kolona) kao i promenljive y, y0, k1, k2, k3.

Primer: rezultati RK 3

```
tkraj = 5.0;
t = (0:1.0:tkraj);
y0 = 1.0;
ya = analitickoResenje.(t,y0);
y1 = Ojler2(difJedn,tkraj,y0,1.0);

t3, y3 = rkutta3(difJedn,0.0,tkraj,[y0],1.0);          # h=1
[collect(t) ya y1 y1-ya y3 y3-ya]
```

```
    t    ya       y1        y1-ya      y3        y3-ya
-----------------------------------------------------------
    0.0  1.0      1.0       0.0        1.0       0.0
    1.0  1.8816   1.86788  -0.0137169  1.87325  -0.00834713
    2.0  2.37421  2.38435   0.0101399  2.36423  -0.00997601
    3.0  2.58626  2.61308   0.0268205  2.5761   -0.0101599
    4.0  2.66895  2.70325   0.0343032  2.65881  -0.0101422
    5.0  2.70003  2.73728   0.0372533  2.68991  -0.0101219
Legenda:
ya  – analitičko rešenje
y1  – Euler 2. reda sa fiksnim korakom: h=1
y3  – RK 3. reda sa fiksnim korakom: h=1
```

RK 4-5. reda

$$k_1 = f(y_i, t_i)h$$

$$k_2 = f\left(y_i + \frac{1}{4}k_1, t_i + \frac{1}{4}h\right)h$$

$$k_3 = f\left(y_i + \frac{1}{32}(3k_1 + 9k_2), t_i + \frac{3}{8}h\right)h$$

$$k_4 = f\left(y_i + \frac{1}{2197}(1932k_1 - 7200k_2 + 7296k_3), t_i + \frac{12}{13}h\right)h$$

$$k_5 = f\left(y_i + \frac{439}{216}k_1 - 8k_2 + \frac{3680}{513}k_3 - \frac{845}{4104}k_4, t_i + h\right)h$$

$$k_6 = f\left(y_i - \frac{8}{27}k_1 + 2k_2 - \frac{3544}{2565}k_3 + \frac{1859}{4104}k_4 - \frac{11}{40}k_5, t_i + \frac{1}{2}h\right)h$$

$$y_{i+1}^{(4)} = y_i + \frac{25}{216}k_1 + \frac{1408}{2565}k_3 + \frac{2197}{4104}k_4 - \frac{1}{5}k_5$$

$$y_{i+1}^{(5)} = y_i + \frac{16}{135}k_1 + \frac{6656}{12825}k_3 + \frac{28561}{56430}k_4 - \frac{9}{50}k_5 + \frac{2}{55}k_6$$

$$e_{i+1}^{(4)} = y_{i+1}^{(5)} - y_{i+1}^{(4)}$$

$$t_{i+1} = t_i + h$$

Implementacija: Runge-Kutta 4-5

$$k_1 = f(y_i, t_i)h$$
$$k_2 = f\left(y_i + \frac{1}{4}k_1, t_i + \frac{1}{4}h\right)h$$
$$k_3 = f\left(y_i + \frac{1}{32}(3k_1 + 9k_2), t_i + \frac{3}{8}h\right)h$$
$$k_4 = f\left(y_i + \frac{1}{2197}(1932k_1 - 7200k_2 + 7296k_3), t_i + \frac{12}{13}h\right)h$$
$$k_5 = f\left(y_i + \frac{439}{216}k_1 - 8k_2 + \frac{3680}{513}k_3 - \frac{845}{4104}k_4, t_i + h\right)h$$
$$k_6 = f\left(y_i - \frac{8}{27}k_1 + 2k_2 - \frac{3544}{2565}k_3 + \frac{1859}{4104}k_4 - \frac{11}{40}k_5, t_i + \frac{1}{2}h\right)h$$

$$y_{i+1}^{(4)} = y_i + \frac{25}{216}k_1 + \frac{1408}{2565}k_3 + \frac{2197}{4104}k_4 - \frac{1}{5}k_5$$
$$y_{i+1}^{(5)} = y_i + \frac{16}{135}k_1 + \frac{6656}{12825}k_3 + \frac{28561}{56430}k_4 - \frac{9}{50}k_5 + \frac{2}{55}k_6$$

$$e_{i+1}^{(4)} = y_{i+1}^{(5)} - y_{i+1}^{(4)}$$
$$t_{i+1} = t_i + h$$

Funkcija vraća niz trenutaka t_k u vektoru T i vrednosti promenljivih stanja y_k kao vektor u svakom od trenutaka iz T. Stoga je Y vektor vektora.

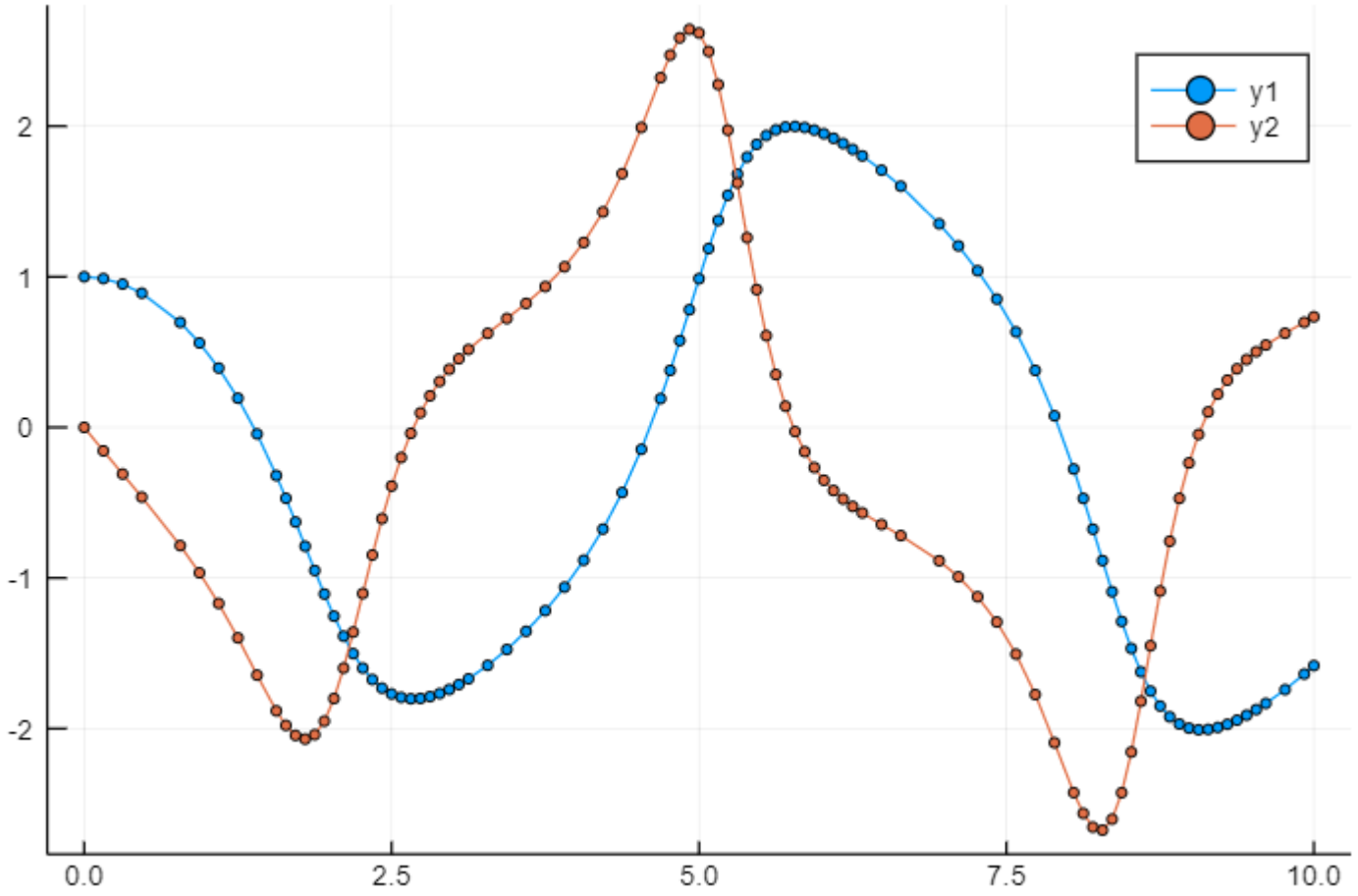
```
function rkutta45( F, tp, tf, y0, hmin, absGr, relGr )
    # Metod integracije RUNGE-KUTTA 4-5 reda, ima promenljivi korak
    t = tp;     T = [t];
    y = y0[:];  Y = [y'];
    h = (tf-tp)/8.0;                  # početni korak
    while t < tf
        if t+h > tf
            h = tf-t;                 # ograniči poslednji korak
        end
        k1 = h * F( t, y );
        k2 = h * F( t+1/4*h, y+1/4*k1 );
        k3 = h * F( t+3/8*h, y+1/32*(3*k1+9*k2) );
        k4 = h * F( t+12/13*h, y+1/2197*(1932*k1-7200*k2+7296*k3) );
        k5 = h * F( t+h, y+439/216*k1-8*k2+3680/513*k3-845/4104*k4 );
        k6 = h * F( t+1/2*h, y-8/27*k1+2*k2-3544/2565*k3+1859/4104*k4-11/40*k5 );
        y4 = y + 25/216*k1 + 1408/2565*k3 + 2197/4104*k4 - 1/5*k5;
        y5 = y + 16/135*k1 + 6656/12825*k3 + 28561/56430*k4 - 9/50*k5 + 2/55*k6;
        e = y5 - y4;
        korakDobar = true;            # fleg za kraj koraka integracije
        if h > hmin                   # Ako je greška velika smanji korak i resetuj fleg
            if all(norm(e) > max(norm(y5)*relGr,absGr))
                h = h / 2.0;
                korakDobar = false;
            end
            if h < hmin
                h = hmin;
            end                       # ograniči min korak
        end
        if korakDobar                 # Nema greške? nastavi ka novoj tački (po vremenu)
            y = y5; t += h;
            push!(Y, y');
            push!(T, t);
            if all(norm(e) < max(norm(y5)*relGr,absGr)/4.0)
                h = h * 2.0;
            end                       # povećati korak?
        end
    end
    return T, Y
end
```

Primer RK 4-5. reda

```
t, y = rkutta45(vdpol, 0.0, 10.0, [1.0;0.0], 1e-6, 1e-6, 1e-6);

using Plots
z = [a[i] for a in y, i in 1:2]
plot(t,z, marker=2)
```

Primer sadrži „prepakivanje“ vrednosti promena promenljivih stanja u matricu z.

```
function vdpol( t, x )
    # Opis Van der Pol-ove dif. jednačine
    x1, x2 = x
    xp = [x2;
          (1-x1^2)*x2-x1 ];
    return xp
end
```



Julija i rešavanje ODJ

- Paket “DifferentialEquations” https://diffeq.sciml.ai/stable/
  - običnih diferencijalnih jednačina - *Ordinary differential equations* (ODEs)
  - diskretnih jednačina (npr. stohastičkih (Gillespie/Markov) simulacija)
  - stohastičkih običnih diferencijalnih jednačina (SODEs ili SDEs)
  - stohastičkih pacijalnih diferencijalnih jednačina ((S)PDEs), i dr.

- Postupak rešavanja ima sledeće korake:
  1. definisanje problema
  2. rešavanje problema
  3. analiza dobijenih rezultata

- Definisanje problema zahteva pisanje funkcije opisa problema (ODJ)

  Razlikuju se opisi:
  1. Jedne ODJ
  2. Sistema ODJ (više jednačina 1. reda)

Upotreba

- uključiti upotrebu prethodno instaliranog paketa

```
using DifferentialEquations
```

- Uvesti promenljivu koja „definiše problem“

```
prob = ODEProblem(f, y0, vremenskiOpseg, p)
```

gde su:

`f` - funkcija koja definiše izvod(e),

`y0` - početne vrednosti zavisno promenljive,

`vremenskiOpseg` - posmatrani vremenski interval,

`p` - parametri modela.

- Numerički „rešiti problem“

```
r = solve(prob)
```

gde je `r` resenje koje sadrži:

`r.t` – vremenski trenuci izračunavanja

`r.u` – odgovarajuće vrednosti zavisno promenljivih

Funkcija f opisuje ODJ

$$\frac{dy}{dt} = f(y, p, t)$$

gde su p parametri modela.

Dodatno, zna se:

$$y(0) = y_0$$

Primer – jedna ODJ

Odrediti rešenje jednačine:

$$\frac{du}{dt} = f(u,p,t) = 4\sin(u), u(0) = 0.5$$

u intervalu $[0, 2]$

```
using DifferentialEquations

f(u,p,t) = 4*sin(u) # funkcija koja opisuje ODJ

u0 = 0.5                          # početna vrednost
tspan = (0.0,2.0)                 # granice vremenskog opsega
                                  #   brojevi su u pokretnom zarezu
prob = ODEProblem(f,u0,tspan)     # definisanje ODJ problema
sol = solve(prob)                 # rešavanje problema
```

```
julia> sol.t
12-element Array{Float64,1}:
 0.0
 0.05949230420978506
 0.1438108087230306
 0.24591650793788414
 0.3811031374915782
 0.5323190660034849
 0.7006811433519459
 0.9191803610910445
 1.1249217212460216
 1.3847234426955484
 1.6699034955734937
 2.0

julia> sol.u
12-element Array{Float64,1}:
 0.5
 0.626554191105659
 0.8521622123844876
 1.1982407654834566
 1.7293895991153825
 2.2698626071966963
 2.6752287277563394
 2.9440170954995897
 3.054593923426414
 3.1107776204536837
 3.1317275654173837
 3.1389443601053832
```

Primer – jedna ODJ i parametari modela

Odrediti rešenje jednačine:

$$\frac{dy}{dt} = \lambda e^{-\alpha t} y, \qquad y(0) = 1$$

u intervalu $[0, 5]$.

Parametri modela su: $\lambda = 1,\ \alpha = 1$

Dodatno: numeričko rešenje se poredi sa analitičkim.

```
using DifferentialEquations
function difJedn(y, p, t)
    α, λ = p
    dy = λ * exp(-α*t) * y
    return dy
end

function analitičkoRešenje(t, y0, p)
    α, λ = p
    y = y0*exp(λ/α*(1-exp(-α*t)))
    return y
end

y0 = 1.0
parametriModela = (1.0, 1.0)
opsegVremena = (0.0, 5.0)
problem = ODEProblem(difJedn, y0, opsegVremena, parametriModela)
r = solve(problem)  # rešenje

referenca(t) = analitičkoRešenje(t, y0, parametriModela)
ytačno = referenca.(r.t)
[r.t r.u ytačno]
```

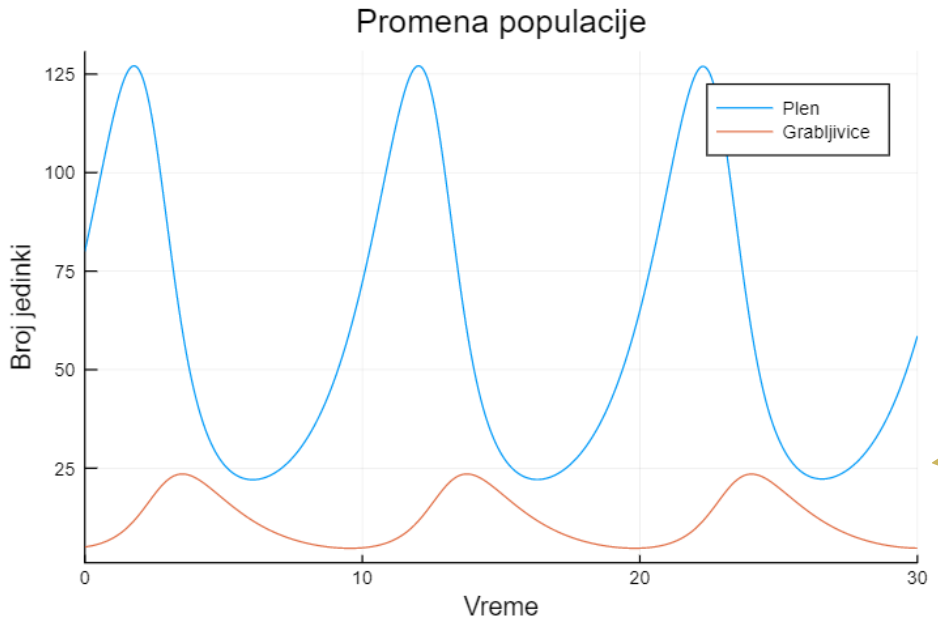
t	y	y_tačno
0.0	1.0	1.0
0.10002	1.09986	1.09986
0.37719	1.36918	1.36918
0.791053	1.72743	1.72743
1.35026	2.09767	2.09767
2.34104	2.46889	2.4689
3.43875	2.63239	2.6324
5.0	2.70002	2.70003

Primer – više ODJ

Lotka-Voter jednačine (promena populacija plena i grabljivica):

$$\frac{dx}{dt} = x(\alpha - \beta y)$$
$$\frac{dy}{dt} = -y(\gamma - \delta x)$$

PAŽNJA: opis ODJ nije isti kao kod jedne ODJ.




```
using DifferentialEquations, Plots

function lotka!(du,u,p,t)
    x, y = u
    α, β, γ, δ = p
    du[1] = x*(α - β*y)
    du[2] = -y*(γ - δ*x)
end

u0 = [80.0; 5.0]                        # početne veličine populacija
tspan = (0.0, 30.0)                     # simulacija 30 meseci
p = [0.7, 0.06, 0.6, 0.01]              # parametri modela
prob = ODEProblem(lotka!,u0,tspan,p)
sol = solve(prob)
plot(sol,title = "Promena populacije",
    lab=["Plen" "Grabljivice"],
    xlabel="Vreme", ylabel="Broj jedinki")
```

Rezultat računanja

iz prethodnog primera

- rešenje sol.u je niz nizova: za svaki trenutak izračunavanja t_i se dobija niz vrednosti zavisno promenljivih $y_1(t_i), \ldots y_n(t_i)$
- Dobijeni rezultati se mogu linearno interpolirati za proizvoljne trenutke (bez ponovnog rešavanja jednačina!)

```
julia> sol(10.0)[1]   # broj zečeva (prva promenljiva) u trenutku 10.0
72.1530609215962

julia> t = [1.0, 2.0, 10.0];     # više trenutaka

julia> sol(t)
t: 3-element Array{Float64,1}:
  1.0
  2.0
 10.0
u: 3-element Array{Array{Float64,1},1}:
 [113.24186748393133, 7.217974737049761]
 [125.47334371129239, 13.568599661677977]
 [72.1530609215962, 4.797360478977869]
```

```
julia> sol.t
30-element Array{Float64,1}:
  0.0
  0.12589379986725807
  0.470150232373384
  0.938805434135219
  1.5149878219178872
...
 27.363027962370886
 28.645561626184755
 30.0

julia> sol.u
30-element Array{Array{Float64,1},1}:
 [80.0, 5.0]
 [84.0886713538732, 5.140642773570113]
 [95.71662719828716, 5.697251072267396]
 [111.35995280575676, 6.990630250013329]
 [125.22056257201332, 9.827974752253033]
...
 [34.277940221857904, 5.772398003074565]
 [58.522907344696954, 4.711468166575054]

julia> sol.t[3]  # 3. trenutak
0.470150232373384

julia> sol.u[3]  # broj zečeva i lisica u t3
2-element Array{Float64,1}:
 95.71662719828716
  5.697251072267396

julia> sol.u[3][2]    # samo broj lisica
5.697251072267396
```

Parametri numeričkog proračuna

(ovo NISU parametri modela!)

- Parametri koji se mogu postaviti su:
  - relativna greška - `reltol` (podrazumevana vrednost je 10^{-3}),
  - apsolutna greška - `abstol` (podrazumevana vrednost je 10^{-6}),
  - frekvenciju snimanja rešenja - `saveat`,
  - pamćenje samo krajnje tačke rešenja (`save_everystep=false`)
  - korak integracije - `dt`, maksimalni korak - `dtmax`, minimalni korak - `dtmin`

  Način postavljanja:

```
r = solve(problem, saveat = 0.01, abstol = 1e-9, reltol = 1e-6)
```

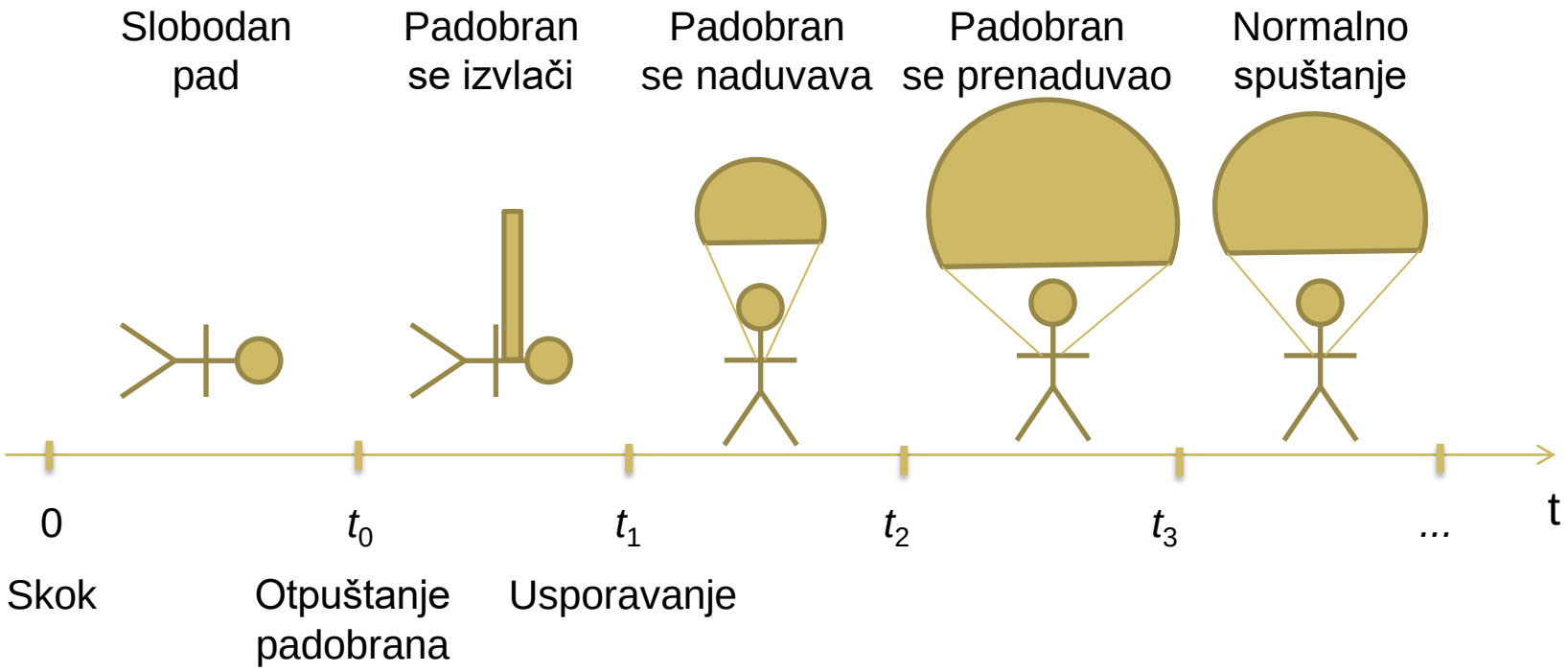
- Funkciji `solve` se može postaviti numerički algoritam za rešavanje ODJ, poput:
  - `BS3()` – varijanta Runge-Kutta 23
  - `DP5()` ili `Tsit5()` – varijante Runge-Kutta 45
  - `Rosenbrock23()` – modifikovan Runge-Kutta 23 za krute probleme, itd.

```
r = solve(problem, Tsit5(), saveat=1.0, abstol=atol, reltol=rtol)
```

Primer - Padobranac

$$m\frac{dv}{dt} + kv^2 = mg, \quad v(0) = 0 \longrightarrow \frac{dv}{dt} = g - \frac{k}{m}v^2$$

$$k = \frac{1}{2}\rho(C_s P_s + C_p P_p)$$



Padobranac - Parametri modela

a_1	b_0	b_1	h	l	m	t_0	t_1	t_2	t_3
43.8 m^2	0.5 m^2	0.1 m^2	1.78 m	8.96 m	97.2 kg	10 s	10.5 s	11,5 s	13.2 s
Površina padobrana	Površina padobranca horizontalna	Površina padobranca vertikalna	Visina padobranca	Dužina kanapa	Težina padobranca	Vreme 0	Vreme 1	Vreme 2	Vreme 3

$$k = \frac{1}{2}\rho \begin{cases} 1.95b_0, & t \le t_0 \\ 1.95b_0 + 0.35b_1 l \dfrac{t - t_0}{t_1 - t_0}, & t_0 < t \le t_1 \\ 0.35b_1 h + 1.33A_1(t), & t_1 < t \le t_2 \\ 0.35b_1 h + 1.33A_2(t), & t_2 < t \le t_3 \\ 0.35b_1 h + 1.33a_1, & t_3 < t \end{cases}$$

$$A_1(t) = \alpha_0 e^{\beta_0 \frac{t - t_1}{t_2 - t_1}}$$

$$A_2(t) = a_1\left(1 + \beta_1 \sin\left(\pi \frac{t - t_2}{t_3 - t_2}\right)\right)$$

$$\alpha_0 = \frac{1.95b_0 + 0.35b_1(l - h)}{1.33}$$

$$\beta_0 = \ln\frac{a_1}{\alpha_0}$$

$$\beta_1 = 0.15$$

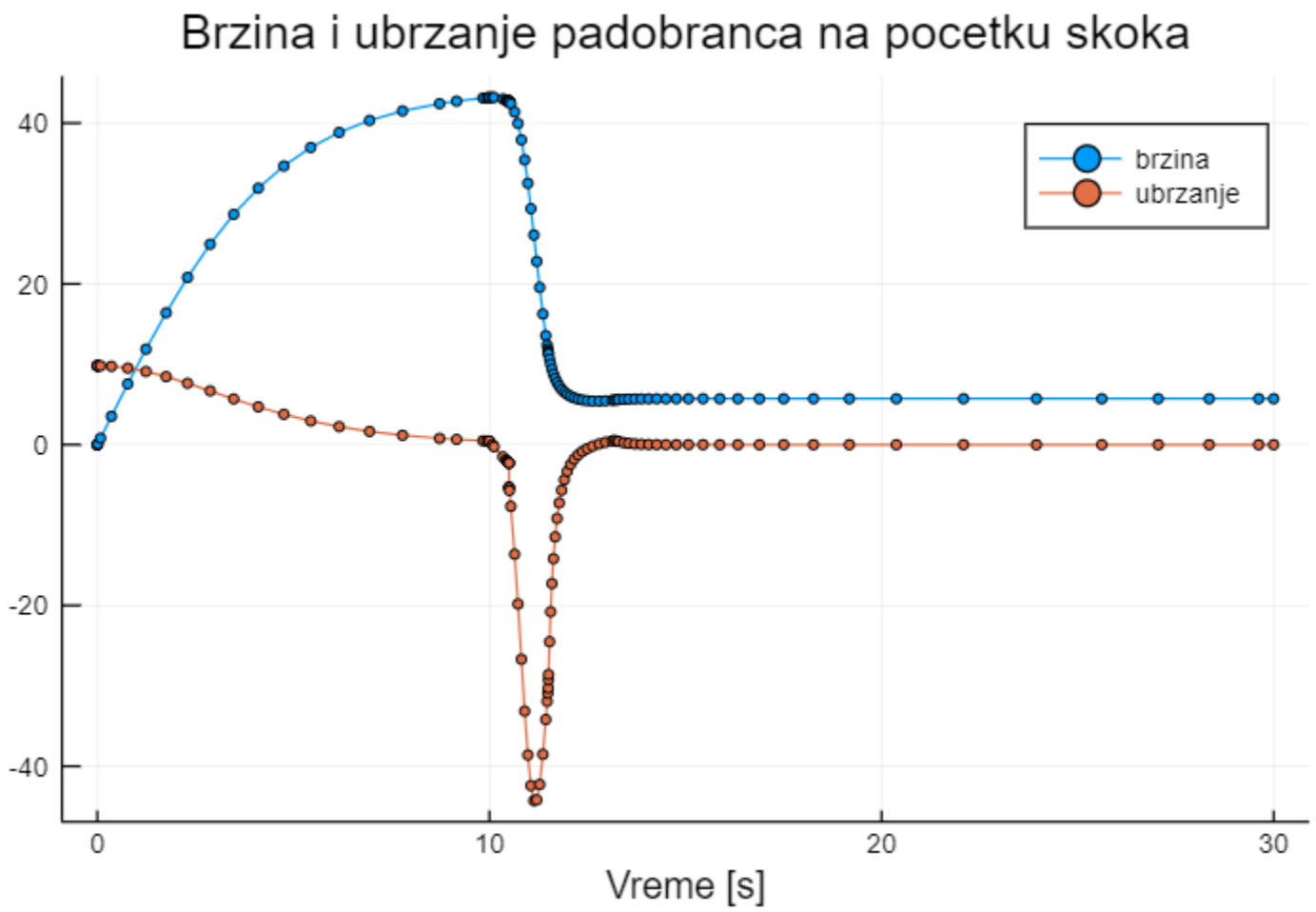
Padobranac - Simulacija

```
function kPadobranca(t, param)
    a₁, b₀, b₁, h, l, m, t₀, t₁, t₂, t₃, β₀, β₁, α₀ = param
    if t <= t₀
        k = 0.5*(1.95*b₀);
    elseif t <= t₁
        k = 0.5*(1.95*b₀+0.35*b₁*l*((t-t₀)/(t₁-t₀)));
    elseif t <= t₂
        A₁ = α₀*exp(β₀*(t-t₁)/(t₂-t₁));
        k = 0.5*(0.35*b₁*h+1.33*A₁);
    elseif t <= t₃
        A₂ = a₁*(1+β₁*sin(π*(t-t₂)/(t₃-t₂)));
        k = 0.5*(0.35*b₁*h+1.33*A₂);
    else
        k = 0.5*(0.35*b₁*h+1.33*a₁);
    end
    return k
end

function ubrzanjePadobranca(v, param, t)
    k = kPadobranca(t, param)
    g = 9.81
    vprim = g - k/m * v^2;
end
```

$$\frac{dv}{dt} = g - \frac{k}{m}v^2$$

```
using DifferentialEquations, Plots

# parametri modela
a₁, b₀, b₁ = (43.8, 0.5, 0.1)
h, l, m = (1.778, 8.96, 97.2)
t₀ = 10.0; t₁ = t₀+0.5; t₂ = t₀+1.5; t₃ = t₀+3.2

β₁, α₀ = 0.15, 1.95*b₀+0.35*b₁*(l-h)/1.33
β₀ = log(a₁/α₀)

# simulacioni interval
interval = (0.0, 30.0)
# početna vrednost brzine
v₀ = 0.0
# formiranje ODE problema i njegovo rešavanje
param = (a₁, b₀, b₁, h, l, m, t₀, t₁, t₂, t₃, β₀, β₁, α₀)
prob = ODEProblem(ubrzanjePadobranca, v₀, interval, param)
r = solve(prob, abstol = 1e-6, reltol = 1e-8)

# racunanje ubrzanja
a = [ubrzanjePadobranca(r.u[i], param,
          r.t[i]) for i = 1:length(r.t)]
# crtanje krivih
plot(r.t,[r.u a],
    title = "Brzina i ubrzanje padobranca na pocetku skoka",
    lab=["brzina" "ubrzanje"], xlabel="Vreme [s]", marker=2)
```

Padobranac - Rezultati simulacije



Analiza rezultata:

- Kolika je vrednost brzine u ustaljenom stanju?
- Koliko traje sila na padobranca >3g?
- Šta se događa kada se padobran otvori nakon 15 sekundi?
- Koliko traje sila na padobranca >3g kada ponese dodatni teret od 30kg?
- …

Neka statistika …

```
using DifferentialEquations

function difJedn(y, p, t)
    α, λ = p
    dy = λ * exp(-α*t) * y
    global brojPoziva += 1
    return dy
end

brojPoziva = 0
y0 = 1.0              # početna vrednost
parametriModela = (1.0, 1.0)
opsegVremena = (0.0, 10.0)

problem = ODEProblem(difJedn, y0,
          opsegVremena, parametriModela)
r = solve(problem)
stat1 = (brojPoziva, length(r.u))

brojPoziva = 0
r2 = solve(problem, saveat=1.0)
stat2 = (brojPoziva, length(r2.u))
```

```
julia> [r.t r.u]
10×2 Array{Float64,2}:
  0.0       1.0
  0.10002   1.09986
  0.37719   1.36918
  0.791053  1.72743
  1.35026   2.09767
  2.34104   2.46889
  3.43875   2.63239
  5.31245   2.70491
  7.16231   2.71617
 10.0       2.71816

julia> t = [1; 2; 3; 9];

julia> [t r(t)]
4×2 Array{Float64,2}:
 1.0  1.88159
 2.0  2.37421
 3.0  2.58625
 9.0  2.71794

julia> stat1
(57, 10)
```

```
julia> [r2.t r2.u]
11×2 Array{Float64,2}:
  0.0  1.0
  1.0  1.88159
  2.0  2.37421
  3.0  2.58625
  4.0  2.66901
  5.0  2.7
  6.0  2.71156
  7.0  2.7158
  8.0  2.71738
  9.0  2.71794
 10.0  2.71816

julia> [t r2(t)]
4×2 Array{Float64,2}:
 1.0  1.88159
 2.0  2.37421
 3.0  2.58625
 9.0  2.71794

julia> stat2
(57, 11)
```

Uticaj zadate tolerancije na računanje

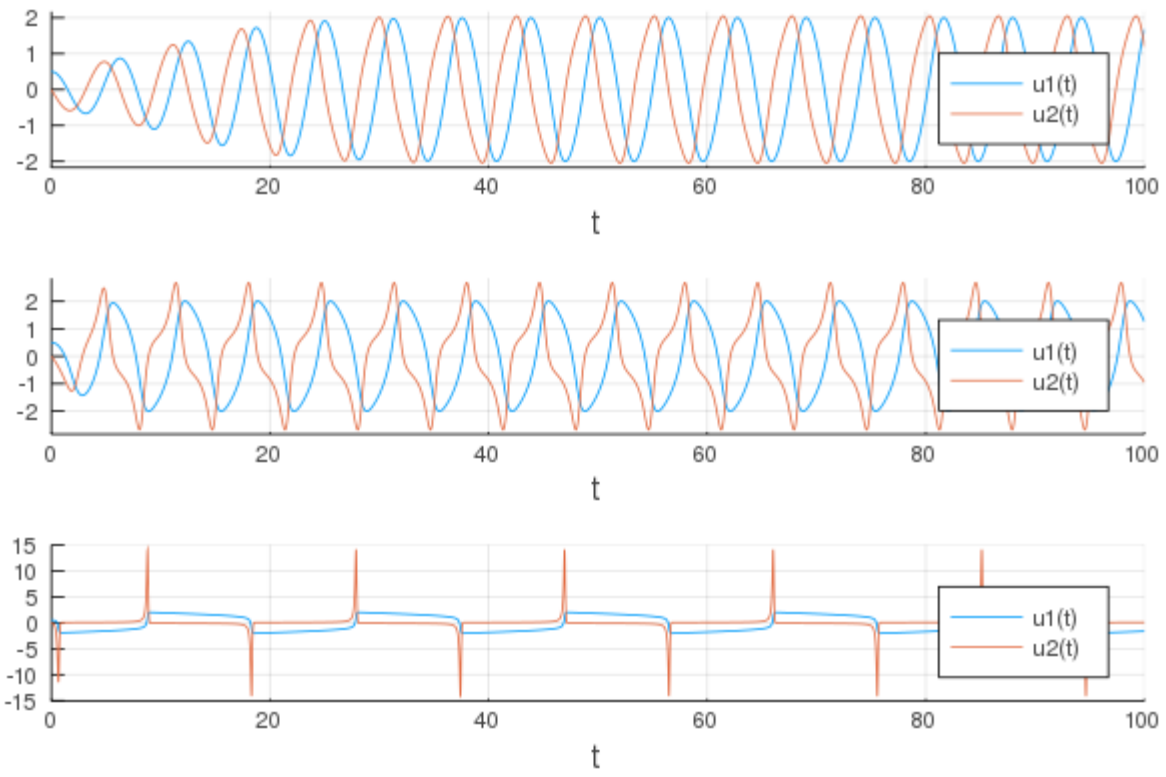
```
for slučaj = 1:4
   # tolerancije računanja
   rtol = atol = 1.0e-02^(slučaj+1)
   # reset brojaca
   global brojPoziva = 0

   problem = ODEProblem(difJedn, y0,
      opsegVremena, parametriModela)
   r = solve(problem, Tsit5(),
      saveat=1.0, abstol=atol, reltol=rtol)

   r1 = r(tref)
   # Prikaz rezultata
   @printf("\nTol = %6.2e\n\n", rtol)
   @printf("   t   ye     y       erry\n")
   for i in 1:length(r1.t)
     @printf("%5.1f %.4f %.4f  %18.15f\n",
        r1.t[i], ytačno[i], r1.u[i],
        ytačno[i]-r1.u[i] )
   end
   @printf("broj poziva = %5d\n", brojPoziva)
end
```

```
Tol = 1.00e-04

   t   ye     y       erry
  0.0 1.0000 1.0000  0.000000000000000
  1.0 1.8816 1.8816  0.000002187605629
  2.0 2.3742 2.3742 -0.000003051268556
  3.0 2.5863 2.5863  0.000002372715780
  4.0 2.6689 2.6689  0.000008711270717
  5.0 2.7000 2.7000 -0.000002760226758
  6.0 2.7116 2.7116  0.000000113235481
  7.0 2.7158 2.7158  0.000000513635455
  8.0 2.7174 2.7174 -0.000000020975642
  9.0 2.7179 2.7179 -0.000000035560870
 10.0 2.7182 2.7182  0.000000259285494
broj poziva =    69

Tol = 1.00e-06

   t   ye     y       erry
  0.0 1.0000 1.0000  0.000000000000000
  1.0 1.8816 1.8816  0.000000235469908
  2.0 2.3742 2.3742  0.000000067247804
  3.0 2.5863 2.5863  0.000000062627066
  4.0 2.6689 2.6689 -0.000000204825310
  5.0 2.7000 2.7000  0.000000340790180
  6.0 2.7116 2.7116  0.000000205946441
  7.0 2.7158 2.7158  0.000000171203649
  8.0 2.7174 2.7174  0.000000154700912
  9.0 2.7179 2.7179 -0.000000032726827
 10.0 2.7182 2.7182  0.000000016523105
broj poziva =   123
```

```
Tol = 1.00e-08

   t   ye     y       erry
  0.0 1.0000 1.0000  0.000000000000000
  1.0 1.8816 1.8816 -0.000000005842887
  2.0 2.3742 2.3742 -0.000000000291064
  3.0 2.5863 2.5863  0.000000011825123
  4.0 2.6689 2.6689  0.000000014591352
  5.0 2.7000 2.7000  0.000000002374046
  6.0 2.7116 2.7116  0.000000003398665
  7.0 2.7158 2.7158  0.000000000847562
  8.0 2.7174 2.7174  0.000000004269482
  9.0 2.7179 2.7179 -0.000000000091185
 10.0 2.7182 2.7182  0.000000002540705
broj poziva =   225

Tol = 1.00e-10

   t   ye     y       erry
  0.0 1.0000 1.0000  0.000000000000000
  1.0 1.8816 1.8816 -0.000000000623472
  2.0 2.3742 2.3742  0.000000000185841
  3.0 2.5863 2.5863  0.000000001215478
  4.0 2.6689 2.6689 -0.000000000001069
  5.0 2.7000 2.7000  0.000000000116953
  6.0 2.7116 2.7116  0.000000000107099
  7.0 2.7158 2.7158 -0.000000000031358
  8.0 2.7174 2.7174  0.000000000001242
  9.0 2.7179 2.7179  0.000000000123423
 10.0 2.7182 2.7182  0.000000000068486
broj poziva =   477
```

Primer: Van der Polova jednačina – još jednom

```
using Plots

function vdpol!( zp, z, p, t )
    x, y = z
    μ = p
    zp[1] = y
    zp[2] = μ * (1-x^2)*y - x
end

interval = (0.0, 100.0)
x0 = [0.5, 0]

prob = ODEProblem(vdpol!, x0, interval, 0.2)
r = solve(prob)
p1 = plot(r)
prob = ODEProblem(vdpol!, x0, interval, 1.0)
r = solve(prob)
p2 = plot(r)
prob = ODEProblem(vdpol!, x0, interval, 10.0)
r = solve(prob)
p3 = plot(r)
plot(p1, p2, p3, layout=(3,1), lw=1)
```



Simulacija ponašanja krugog (engl. *stiff*) problema

Posmatramo model amortizera ...

Kruti problem opisuje model gde se svojstvene vrednosti (realni delovi) razlikuju za nekoliko redova veličina. Posledica je da se neke promenljive stanja sporo menjaju (male svojstvene vrednosti), a druge brzo, što nekim numeričkim algoritmima za rešavanje ODJ otežava prilagođavanje koraka integracije. Postoje posebni algoritmi pogodni na takve probleme.

```
julia> using Polynomials;

julia> p = Polynomial([10,11,1], :s)
Polynomial(10 + 11*s + s^2)

julia> roots(p)
2-element Array{Float64,1}:
 -10.0
  -1.0

julia> p = Polynomial([1000,1001,1], :s)
Polynomial(1000 + 1001*s + s^2)

julia> roots(p)
2-element Array{Float64,1}:
 -1000.0
    -1.0
```

```
Trajanje   Brojač Dužina
------------------------
0.000146     195      64
0.002699   12012    4004
0.000790     338      68
```

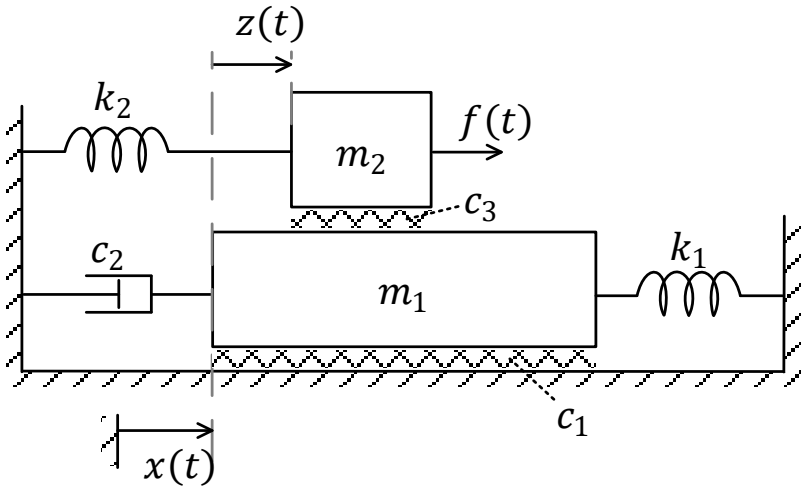
```
function oscil!(xp, x, parametri, t)
    (m, c, k) = parametri
    global brojac += 1;
    xp[1] = x[2]
    xp[2] = (-c*x[2] - k*x[1])/m
end

@printf("Trajanje   Brojač Dužina\n")
@printf("------------------------\n")
x0 = [1.0, 0]
interval = (0, 10.0)

brojac = 0;
parametri = (1, 11, 10);      # m,c,k
prob = ODEProblem(oscil!, x0, interval, parametri);
s = @elapsed r = solve(prob, BS3());
@printf("%8.6f  %6d  %6d\n", s,brojac,length(r.t))

brojac = 0;
parametri = (1, 1001, 1000);  # m,c,k - krut model
prob = ODEProblem(oscil!, x0, interval, parametri);
s = @elapsed r = solve(prob, BS3());
@printf("%8.6f  %6d  %6d\n", s,brojac,length(r.t))

brojac = 0;
parametri = (1, 1001, 1000);  # m,c,k
prob = ODEProblem(oscil!, x0, interval, parametri);
s = @elapsed r = solve(prob, Rosenbrock23());
@printf("%8.6f  %6d  %6d\n", s,brojac,length(r.t))
```

Poredjenje "naše" implementacije RK45 sa rešenjem u paketu

```
function vdpol( t, x )
    # Opis Van der Pol-ove dif. jednacine
    x1, x2 = x
    xp = [x2;
          (1-x1^2)*x2-x1 ];
    return xp
end

function vdpol!( zp, z, p, t )
    x, y = z
    μ = p
    zp[1] = y
    zp[2] = μ * (1-x^2)*y - x
end

t, y = rkutta45(vdpol, 0.0, 10.0, [1.0;0.0], 1e-6, 1e-6, 1e-6);

interval = (0.0, 10.0)
x0 = [1.0, 0]
prob = ODEProblem(vdpol!, x0, interval, 1.0)
r = solve(prob, Tsit5(), abstol=1e-6, reltol=1e-6)

@printf("%10.6f   %10.6f\n", y[end][1], y[end][2])
@printf("%10.6f   %10.6f\n", r.u[end][1], r.u[end][2])
```

```
-1.582034     0.734183
-1.582030     0.734184
```

Primer mehaničkog sistema




```
function mehSistem(dy, y, param, t)
    M₁, M₂, c, c₃, k₁, k₂, pobuda = param
    dy[1] = y[2]
    dy[2] = 1/M₁*(c₃*y[4]-c*y[2]-k₁*y[1] )
    dy[3] = y[4]
    dy[4] = 1/M₂*(pobuda(t)-c₃*y[4]-k₂*(y[1]+y[3]))-dy[2]
end
```

U kodu je koef. trenja $c = c_1 + c_2$

Promenljive stanja su:

$$y_1 = x$$

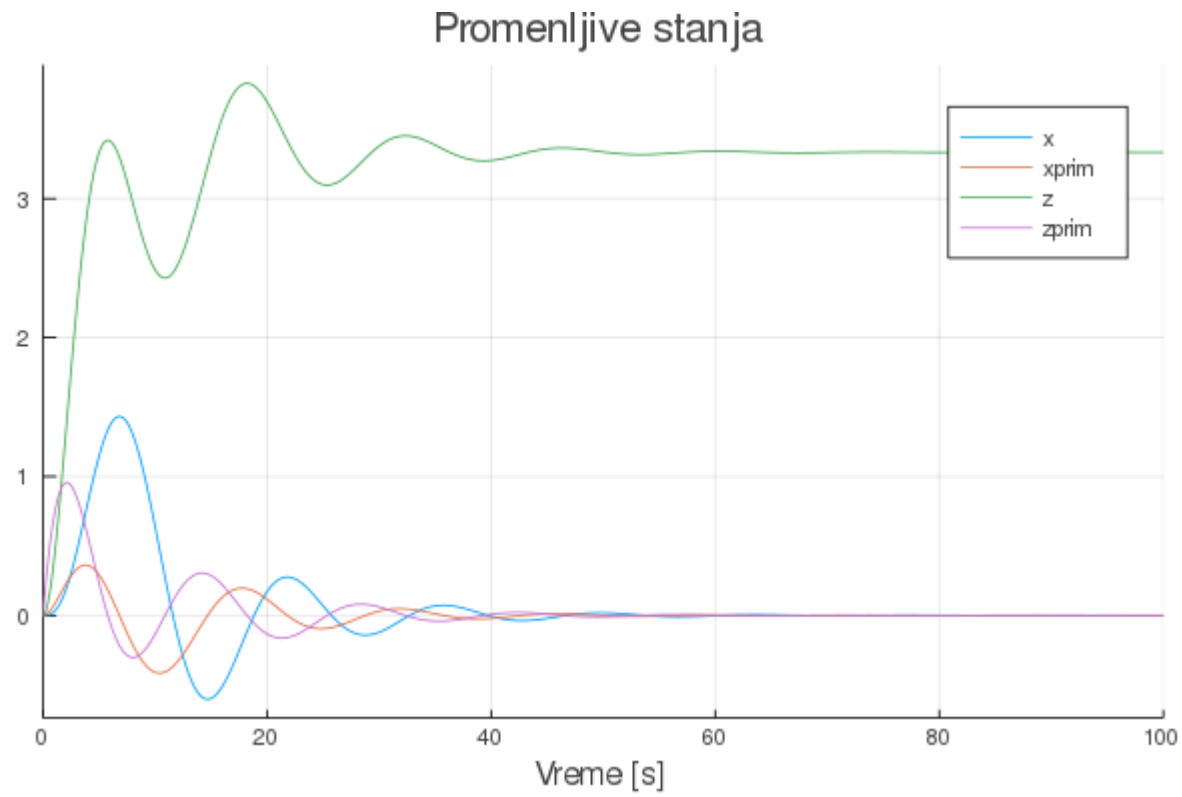
$$y_2 = \dot{x}$$

$$y_3 = z$$

$$y_4 = \dot{z}$$

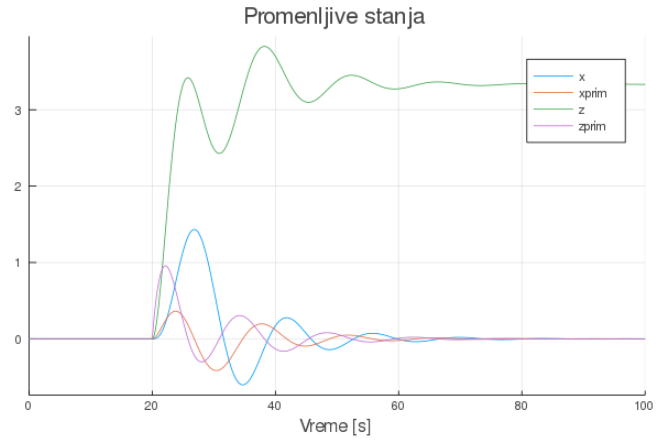
Mehanički sistem (2)

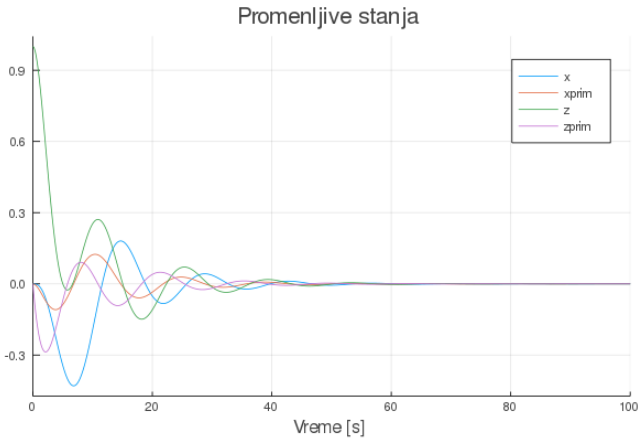
```
using DifferentialEquations, Plots

M₁, M₂, c, c₃, k₁, k₂ = (2.0, 1.0, 0.3, 0.4, 0.2, 0.3)
interval = (0.0, 100.0)
y₀ = [0.0, 0.0, 0.0, 0.0]

# pobuda je 1
parametri = (M₁, M₂, c, c₃, k₁, k₂, t -> 1.0)
prob = ODEProblem(mehSistem, y₀, interval, parametri)
r = solve(prob)

plot(r, title = "Promenljive stanja",
    lab=["x" "xprim" "z" "zprim"], xlabel="Vreme [s]")
```

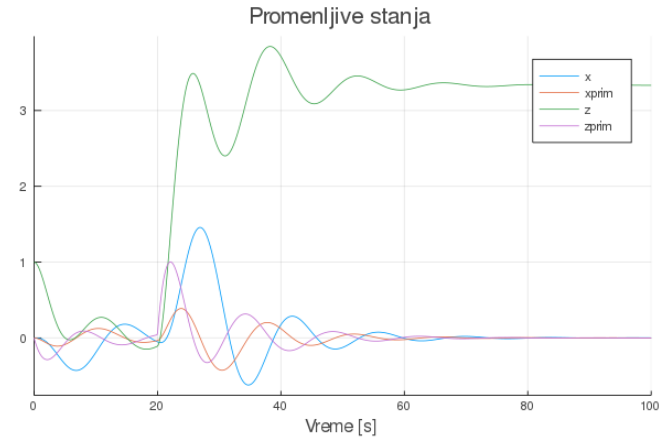


Mehanički sistem (2)

```
# poduda kasni, početne vrednosti su nula
y₀ = [0.0, 0.0, 0.0, 0.0]
parametri = (M₁, M₂, c, c₃, k₁, k₂, t -> (t > 20) ? 1.0 : 0.0)
prob = ODEProblem(mehSistem, y₀, interval, parametri)
r1 = solve(prob)

plot(r1, title = "Promenljive stanja",
    lab=["x" "xprim" "z" "zprim"], xlabel="Vreme [s]")

# početna vrednost nije nula, ali pobuda jeste
y₀ = [0.0, 0.0, 1.0, 0.0]
parametri = (M₁, M₂, c, c₃, k₁, k₂, t -> 0.0)
prob = ODEProblem(mehSistem, y₀, interval, parametri)
r2 = solve(prob)

plot(r2, title = "Promenljive stanja",
    lab=["x" "xprim" "z" "zprim"], xlabel="Vreme [s]")

# početne vrendosti i pobuda nisu nula
y₀ = [0.0, 0.0, 1.0, 0.0]
parametri = (M₁, M₂, c, c₃, k₁, k₂, t -> (t > 20) ? 1.0 : 0.0)
prob = ODEProblem(mehSistem, y₀, interval, parametri)
r3 = solve(prob)

plot(r3, title = "Promenljive stanja",
    lab=["x" "xprim" "z" "zprim"], xlabel="Vreme [s]")
```

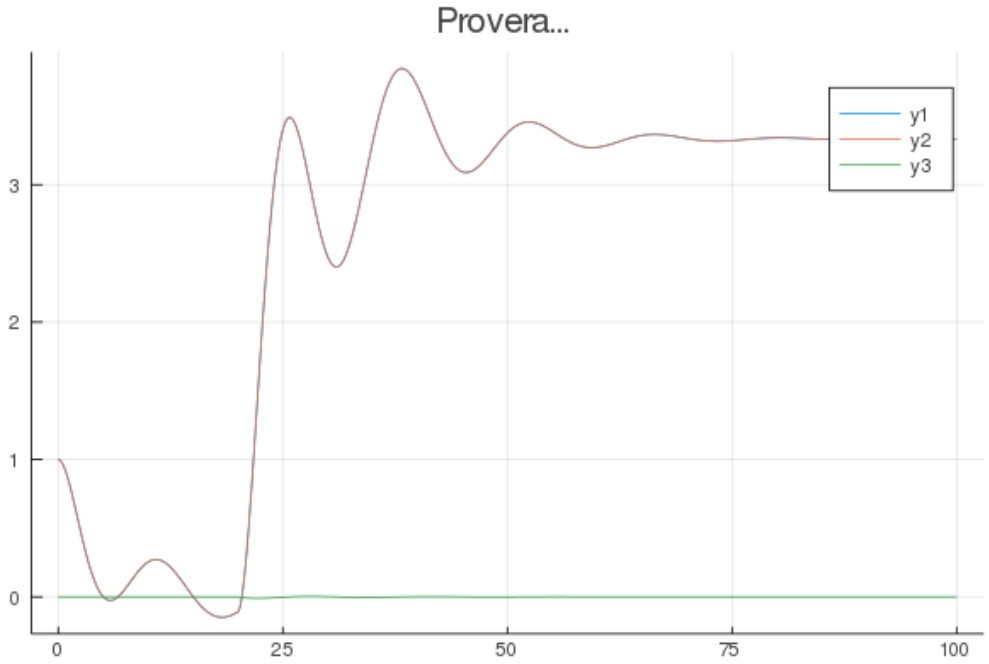






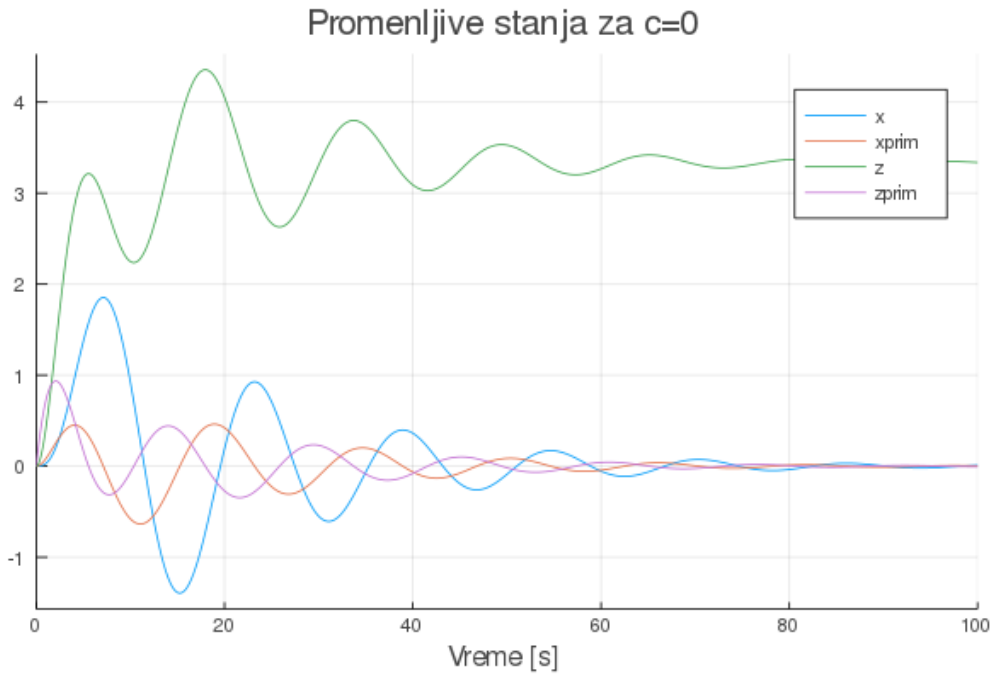
Mehanički sistem (3)

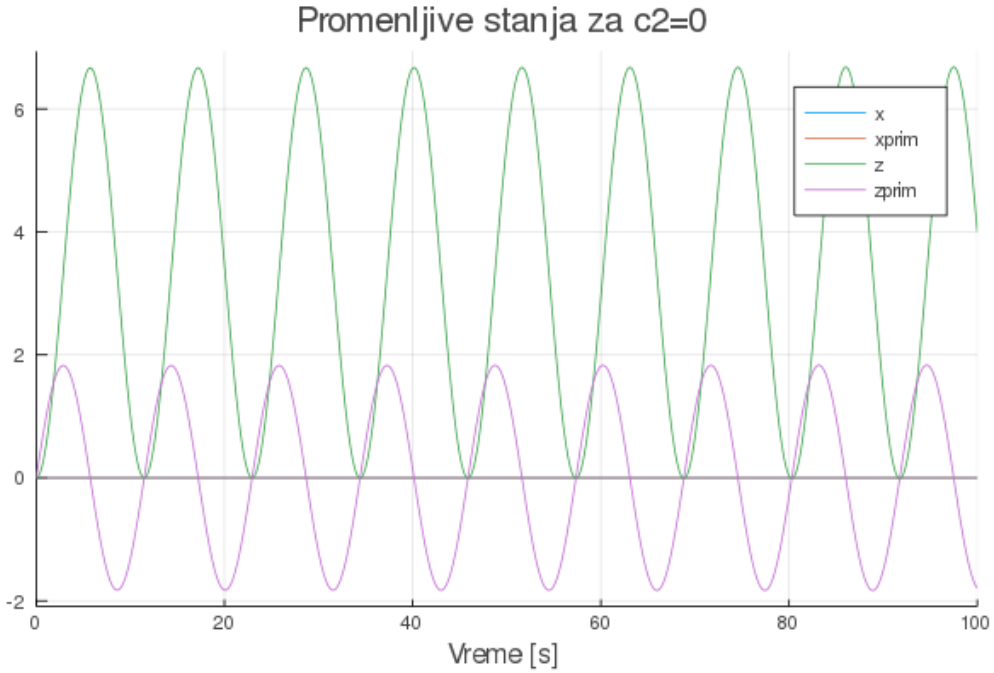
```
# uporedni prikaz
t = 0:0.1:100;                              # vremska osa
z1 = [x[3] for x in r1(t).u]
# ili: z1 = [r1(t).u[i][3] for i=1:length(t)]
z2 = [x[3] for x in r2(t).u]
z3 = [x[3] for x in r3(t).u]
plot(t, [z1 z2 z3],
          title="Uporedni prikaz z(t) iz 3 scenarija")
plot(t, [(z1.+z2) z3 (z1.+z2.-z3)],
          title="Provera...")
```



Mehanički sistem (4)

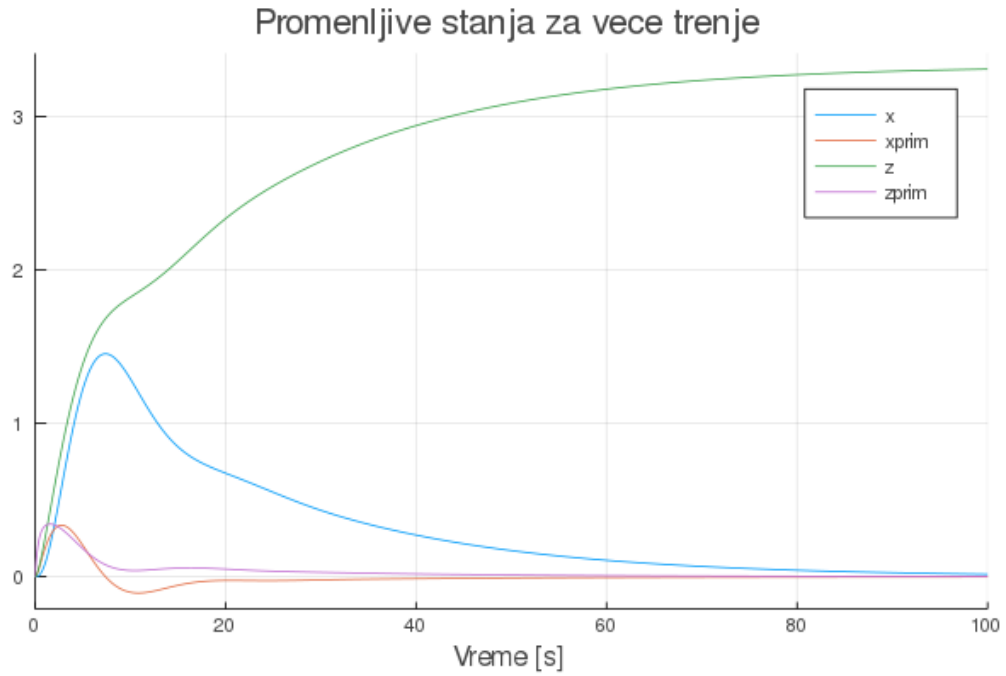
```
# isti model, ali nema trenja c
y₀ = [0.0, 0.0, 0.0, 0.0]
parametri = (M₁, M₂, 0, c₃, k₁, k₂, t -> 1.0)
prob = ODEProblem(mehSistem, y₀, interval, parametri)
r = solve(prob)
plot(r, title = "Promenljive stanja za c=0",
    lab=["x" "xprim" "z" "zprim"], xlabel="Vreme [s]")

# isti model, ali nema trenja c₃
y₀ = [0.0, 0.0, 0.0, 0.0]
parametri = (M₁, M₂, c, 0, k₁, k₂, t -> 1.0)
prob = ODEProblem(mehSistem, y₀, interval, parametri)
r = solve(prob)
plot(r, title = "Promenljive stanja za c2=0",
    lab=["x" "xprim" "z" "zprim"], xlabel="Vreme [s]")
```





Mehanički sistem (5)

```
# Šta se događa sa sistemom kada je trenje veće?
y₀ = [0.0, 0.0, 0.0, 0.0]
parametri = (M₁, M₂, 5c, 5c₃, k₁, k₂, t -> 1.0)
prob = ODEProblem(mehSistem, y₀, interval, parametri)
r = solve(prob)
plot(r, title = "Promenljive stanja za vece trenje",
    lab=["x" "xprim" "z" "zprim"], xlabel="Vreme [s]")
```



Mehanički sistem (6)

```
# Neka složenija pobuda ...
function složenijaPobuda(t)
    tp = t % 50
    if tp <= 10
        return tp / 10
    elseif tp <= 20
        return 1
    elseif tp <= 30
        return 1 - (tp-20)/10
    else
        return 0;
    end
end

t = 0:0.1:100
plot(t, složenijaPobuda.(t), lab="pobuda")

y₀ = [0.0, 0.0, 0.0, 0.0]
parametri = (M₁, M₂, c, c₃, k₁, k₂, složenijaPobuda)
prob = ODEProblem(mehSistem, y₀, interval, parametri)
r = solve(prob)
plot(r,
    title = "Promenljive stanja za slozeniju pobudu",
    lab=["x" "xprim" "z" "zprim"], xlabel="Vreme [s]")
```